

Hashcat — Study Guide

Hashcat is the tool your John the Ripper guide flagged as "generally considered the more mature/faster tool specifically for GPU cracking" — this guide picks up exactly there. Same underlying model as JtR (hash a candidate, compare to target, repeat at scale), but Hashcat is built **GPU-first from the ground up** rather than having GPU support bolted on, which changes not just raw speed but the entire mental model around attack modes, numbering, and syntax. Worth locking in the distinction immediately: **Part 1 covers Hashcat's core attack modes** (numbered `-a` modes — its own naming convention, distinct from JtR's mode names) — **Part 2 covers performance/advanced techniques** (rules, hybrid attacks, benchmarking, and distributed cracking at scale) — the same "what to try" vs "how to try it smarter/faster" split your JtR guide used, now optimized around a GPU-native architecture instead of a CPU-native one.

1. Why Hashcat and JtR Aren't Just "The Same Tool Twice" — The Foundation

Both tools solve the identical underlying problem (candidate generation → hash → compare), but the **architecture assumption** baked in from day one differs fundamentally, and that single design choice cascades into almost every practical difference you'll notice:

John the Ripper → CPU-native by default, GPU support added later via jumbo/OpenCL for mats (your JtR guide, Section 11)

Hashcat → GPU-native from inception - kernel design, candidate generation, and memory access patterns are all built assuming massively parallel hardware from the start

The single fact that explains almost every speed comparison you'll see cited — for fast, parallelizable hashes (NTLM, raw MD5/SHA), Hashcat on a decent GPU will typically outperform JtR by a wide margin; for slow/memory-hard hashes (bcrypt, Argon2) specifically engineered to resist parallelization, the gap narrows dramatically, since the algorithm itself is the bottleneck, not which tool is driving it. This is why practitioners commonly run both: JtR for its broader odd-format coverage, single-crack mode, and CPU flexibility; Hashcat for large-scale GPU wordlist/mask runs against common fast-hash formats.

PART 1: Core Attack Modes

2. Straight Mode (**a 0**) — Dictionary Attack

Hashcat's equivalent of JtR's wordlist mode (your JtR guide, Section 3) — every line in a wordlist is hashed and compared directly, optionally combined with a rule file:

```
hashcat -a 0 -m 0 hashes.txt rockyou.txt
hashcat -a 0 -m 0 hashes.txt rockyou.txt -r best64.rule
```

```
-a 0 → attack mode: straight (dictionary)
-m 0 → hash mode: 0 = raw MD5 (Section 8 covers hash mode numbers)
```

Same effectiveness principle as JtR's wordlist mode: output quality is overwhelmingly a function of **wordlist relevance**, not the tool driving it — rockyou.txt remains the default starting point for the same breach-derived-realism reasons covered in your JtR guide.

3. Combinator Mode (**a 1**) — Concatenating Two Wordlists

Takes every word from one list and appends every word from a second list, generating the full cross-product as candidates — useful when you suspect passwords are built from two combined real-word components:

```
hashcat -a 1 -m 0 hashes.txt wordlist1.txt wordlist2.txt
```

```
wordlist1.txt: dragon, tiger, phoenix
wordlist2.txt: castle, fortress
```

```
Generated candidates:
dragoncastle, dragonfortress,
tigercastle, tigerfortress,
phoenixcastle, phoenixfortress
```

4. Brute-Force / Mask Attack (**a 3**)

Hashcat's direct equivalent of JtR's mask mode (your JtR guide, Section 9) — defines which character class occupies which position, dramatically narrowing

the search space compared to blind incremental brute-force:

```
hashcat -a 3 -m 0 hashes.txt ?u?l?l?l?l?d?d?d?d
```

```
?l = lowercase    ?u = uppercase    ?d = digit    ?s = special  
?a = all printable ASCII characters
```

Mask ?u?l?l?l?l?d?d?d?d matches "Snake2024"-style structure - identical concept to JtR's mask syntax, same underlying logic, different tool's flag conventions

Hashcat additionally supports `?1` / `?2` / `?3` / `?4` **custom charsets**, letting you define your own character class per position rather than only using the four built-in ones:

```
hashcat -a 3 -m 0 hashes.txt -1 '?l?d' ?1?1?1?1?1?1  
# custom charset "?1" = lowercase+digits, tries all 6-char  
# combinations from that reduced set
```

5. Hybrid Attacks (`a 6` and `a 7`) — Wordlist + Mask Combined

Hashcat's equivalent of JtR's hybrid mode (your JtR guide, Section 10), split into two distinct directional modes:

```
hashcat -a 6 -m 0 hashes.txt rockyou.txt ?d?d?d?d # wordlist + APPENDED mask  
hashcat -a 7 -m 0 hashes.txt ?d?d?d?d rockyou.txt # mask PREPENDED + wordlist
```

```
-a 6: dragon + ?d?d?d?d → dragon0000, dragon0001, ... dragon9999  
-a 7: ?d?d?d?d + dragon → 0000dragon, 0001dragon, ... 9999dragon
```

Splitting this into two explicit directional modes (rather than JtR's single `?w` placeholder approach) is a small but real syntax difference worth knowing before your first real engagement, so you're not debugging mode selection under time pressure.

6. Association Attack (`a 9`)

A less commonly used but genuinely distinct mode with no direct JtR equivalent — pairs each candidate password specifically with its corresponding hash's associated metadata (useful for formats where the hash itself embeds a username or salt that should inform the guess, such as certain Office document or PDF encryption formats where the filename/metadata is a plausible password component).

PART 2: Performance & Advanced Techniques

7. Benchmarking Your Hardware First

Before committing to a long-running attack, benchmark to understand realistic throughput for the hash type you're targeting — this directly determines whether incremental/mask attacks against a given search space are even remotely feasible in available time:

```
hashcat -b # benchmark all supported hash modes
hashcat -b -m 1000 # benchmark NTLM specifically
```

Typical relative throughput pattern on a modern GPU:

Raw MD5 / NTLM	→ tens of billions of hashes/second
SHA-256	→ billions of hashes/second
bcrypt	→ thousands to low tens of thousands of hashes/second (by design)

This ~6-7 order-of-magnitude gap is Section 1's core lesson made concrete in real numbers - it's why a mask attack against 8 characters is trivial for NTLM and effectively infeasible for bcrypt on the same hardware.

8. Hash Mode Identification (`m`)

Hashcat identifies algorithms via **numeric hash mode codes** rather than JtR's named `--format` strings — the same Section 7 lesson from your JtR guide (never guess the format) applies here, just with a different lookup mechanism:

```
hashcat --help | grep -i ntlm # search for a mode by keyword
```

```
hashcat --example-hashes -m 1000 # show example hash + confirm format
```

Common hash mode numbers:

- 0 → raw MD5
- 100 → raw SHA-1
- 1000 → NTLM
- 1800 → sha512crypt (common modern Linux /etc/shadow)
- 3200 → bcrypt
- 13100 → Kerberos 5 TGS-REP (Kerberoasting - directly relevant if you're chaining this from an AD engagement)
- 22000 → WPA-PBKDF2-PMKID+EAPOL (modern WiFi handshake cracking, superseding the older 2500 mode)

A wrong `-m` value is the single most common reason a Hashcat session runs to completion with zero cracks despite a perfectly good wordlist — identical failure mode to JtR's `--format` mismatch, just under a different flag name.

9. Rule-Based Attacks in Depth

Hashcat ships with the same style of rule files referenced in your JtR guide (`best64.rule` is available in both tools, cross-compatible in large part) plus its own larger community rule sets:

```
hashcat -a 0 -m 0 hashes.txt rockyou.txt -r rules/best64.rule
hashcat -a 0 -m 0 hashes.txt rockyou.txt -r rules/OneRuleToRuleThemAll.rule
```

Rule syntax overlaps heavily with JtR's (Section 8 of your JtR guide) since both draw from the same historical rule-syntax lineage:

- c → capitalize first letter
- \$1 → append the digit 1
- sa@ → substitute a with @
- r → reverse the whole word

`OneRuleToRuleThemAll` and similar large community rule sets trade runtime for coverage — genuinely large rule files can turn a modest wordlist into an effectively enormous candidate space, at a real time cost worth budgeting for.

10. Distributed / Multi-Device Cracking

Hashcat natively splits work across **every GPU (and CPU) in a single machine** automatically, and supports splitting a job across **multiple machines** via manual keyspace division:

```
hashcat -a 3 -m 0 hashes.txt ?a?a?a?a?a?a?a?a --skip=0 --limit=10000000000
# Machine 2 picks up where Machine 1 left off:
hashcat -a 3 -m 0 hashes.txt ?a?a?a?a?a?a?a?a --skip=1000000000 --limit=10000000000
```

```
-O → enable optimized kernels (faster, but restricts max password length per hash type - fine for most real-world cases)
-w 3 → workload profile (aggressive GPU utilization, may impact desktop responsiveness on the cracking machine)
```

For genuinely large-scale distributed cracking across a cluster, dedicated orchestration tools (Hashtopolis is the most common) sit on top of Hashcat to automatically divide keyspace and aggregate results across many nodes, rather than manually managing `--skip` / `--limit` ranges by hand.

11. Potfile and Session Management

```
hashcat -a 0 -m 0 hashes.txt rockyou.txt --session=engagement1
hashcat --session=engagement1 --restore
hashcat --show hashes.txt # show cracked results from the potfile
```

Hashcat automatically records every cracked hash:plaintext pair into a "potfile" (hashcat.potfile) so already-cracked hashes are never redundantly re-attempted across sessions - functionally equivalent to JtR's `--show` mechanism, but persisted automatically by default rather than needing explicit re-invocation

12. Testing / Engagement Methodology

1. Identify the hash mode precisely (Section 8) - confirm via `--example-hashes` before committing to a long run

2. Benchmark (Section 7) against the relevant mode to set realistic expectations for what's actually feasible in your engagement window
3. Run straight/dictionary mode with rules FIRST (Sections 2, 9) against a breach-relevant wordlist - highest general value per your JtR guide's identical Section 3 principle
4. Layer in hybrid/mask attacks (Sections 4-5) if organizational password structure is known or suspected
5. For very large jobs, consider distributing across available hardware (Section 10) rather than running serially
6. Track cracked-vs-total and TIME-to-crack per password as reportable engagement metrics, same as your JtR guide's Section 13 point - this concrete data is what actually drives remediation prioritization conversations with a client

13. Legal & Ethical Note

Identical to your JtR guide's Section 14: cracking hashes without explicit, documented authorization (a signed engagement scope, your own lab, or a hash you legitimately own) is illegal in most jurisdictions regardless of tool or technique. Every example here assumes an authorized penetration test, CTF/lab environment, or personal credential recovery.

14. Prevention

Identical root-cause defenses to your JtR guide's Section 15 apply here unchanged, since Hashcat and JtR attack the exact same underlying weakness (fast/unsalted hashing, weak password policy, credential reuse) - the tool doesn't change what defends against it:

- Use bcrypt/scrypt/Argon2id for password storage, never a fast general-purpose hash
- Always use a unique, random per-user salt
- Favor length over forced complexity patterns (Section 4's mask attack directly targets predictable complexity-driven structures)
- Screen new passwords against known-breached corpora at signup
- Rate-limit/lock out ONLINE authentication attempts (irrelevant to offline cracking of an already-dumped hash, but closes the parallel path)

15. Practical Lab Example

TryHackMe and Hack The Box both have Hashcat-specific rooms covering hash mode identification and mask/rule construction against realistic dumps; several rooms explicitly pair a Kerberoasting or WiFi-handshake-capture walkthrough with the resulting hashcat -m 13100 / -m 22000 cracking step, which is a genuinely good way to see the FULL kill chain from capture to plaintext in one exercise rather than practicing cracking in isolation.

Suggested practice order:

1. Benchmark your own hardware first (Section 7) - know your realistic throughput before running anything real
2. Run straight mode + best64 rules against a small raw-md5 test set (Sections 2, 9) - confirm syntax and workflow
3. Build and time a mask attack (Section 4) against a synthetic password set you generate yourself following a known structure - directly compare the time-to-crack against your JtR guide's equivalent Section 9 exercise, on the same hardware, to make Section 1's architecture difference concrete rather than theoretical

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-916 (Use of Password Hash With Insufficient Computational Effort), CWE-521 (Weak Password Requirements) - identical mapping to your JtR guide, since both tools exploit the same storage-side root cause
OWASP Top 10:2025	A02:2025 Security Misconfiguration / Cryptographic Failures territory (algorithm choice, salting)
CVSS	Not directly CVSS-scored (a tool, not a vulnerability) - severity inherits from whatever exposed the hash dump initially
Cyber Kill Chain	Actions on Objectives (cracking converts an already-obtained hash into a usable credential for further lateral movement, persistence, or privilege escalation)
MITRE ATT&CK	T1110.002 (Brute Force: Password Cracking), T1003 (OS Credential Dumping, the typical preceding step) - same mapping as JtR, since ATT&CK classifies by technique/objective, not by which tool executes it

Quick Reference Cheat Sheet

ATTACK MODES (-a):

- a 0 → straight/dictionary (JtR: --wordlist)
- a 1 → combinator (two wordlists combined) (no direct JtR mode)
- a 3 → brute-force / mask (JtR: --mask / --incremental)
- a 6 → wordlist + appended mask (JtR: --mask='?w...')
- a 7 → mask + appended wordlist (JtR: reverse hybrid)
- a 9 → association attack (no direct JtR mode)

ESSENTIAL FLAGS:

- m X → hash mode number (CHECK THIS FIRST, always)
- r FILE.rule → apply mangling rules
- b → benchmark
- example-hashes → confirm a hash mode's expected format
- session=NAME → name a resumable session
- show → display cracked results from potfile

HASHCAT vs JOHN THE RIPPER – quick translation:

- JtR --format=X ↔ Hashcat -m NUMBER
- JtR --wordlist=F ↔ Hashcat -a 0 F
- JtR --mask='?u?l?d' ↔ Hashcat -a 3 ?u?l?d
- JtR --rules=SET ↔ Hashcat -r FILE.rule
- JtR CPU-native, GPU bolted-on ↔ Hashcat GPU-native from inception

Root lesson (unchanged from JtR guide): cracking speed is a function of (hash algorithm cost) x (candidate quality) x (parallelization the hardware/tool can exploit) - Hashcat mainly moves the THIRD variable, not the first two, which is why bcrypt stays resistant even here.